First Project

Sistemas Operativos

Students:

Manfred Azofeifa Salazar
Jose Adrián Herrera Salas

Professor:
Eddy Ramírez Jiménez

March 24, 2026

Contents

1 Introduction **2**
1.1 Main Objective of the asignation 2
1.2 Characteristics of the parts of the project 2
1.3 Modes 2

2 Theoretical Framework **3**

3 Explanation of the Solution **4**
3.1 Architecture Overview 4
3.2 Bridge Synchronization 4
3.3 Bridge Traversal 4
3.4 Control Modes 5
3.5 Ambulance Handling 5
3.6 Visualization 5

4 Description of the Experience **6**
4.1 Problems encountered 6
4.2 "Unique" Decisions 6

5 Personal Lessons Learned **7**
5.1 Manfred's Lessons 7
5.2 Jose's Lessons 7

6 Conclusions **8**

1 Introduction

1.1 Main Objective of the asignation

The main objective of this project was to develop a deeper understanding of how an operating system works, through the implementation of a narrow bridge simulation. A complete implementation required applying the following concepts covered in the course:

1. Basic concepts such as system calls and process management.
2. How the operating system manages concurrent processes and shared resources.

1.2 Characteristics of the parts of the project

The main characteristic of the bridge is that it is only wide enough for one car at a time, meaning two cars traveling in opposite directions cannot occupy it simultaneously. The simulation supports multiple control modes that determine how cars are allowed to cross.

Before describing each mode, it is important to note that there are two types of vehicles: regular cars and ambulances. Ambulances are granted special privileges that allow them to cross as quickly as possible; the exact nature of these privileges varies depending on the active mode and will be detailed in each respective description.

1.3 Modes

The first mode is **Carnage**. In this mode, the only restriction preventing a car from entering the bridge is if a car traveling in the opposite direction is already on it. If the bridge is empty, any car may enter freely. If an ambulance is first in the waiting queue, no car from the opposing lane may enter, and the ambulance will cross as soon as the bridge is clear.

The second mode uses **traffic lights**. Each lane has its own traffic light. One side starts green and, after a configurable number of seconds, switches to red while the opposing side turns green. In this mode, if an ambulance is at the front of a waiting lane, no cars from the opposing lane will be allowed to enter, and the ambulance will cross as soon as the bridge is empty, regardless of the current light state.

The third and final mode is controlled by a **police officer**. The officer allows a fixed number of cars from one lane to cross, then switches priority to the other lane for the same quota, alternating continuously. If an ambulance is first in line on the waiting side and no car is currently crossing in the opposite direction, the officer will allow it to pass immediately without altering the current turn.

Some variables of the bridge, mode and car all going to be taken for a different

archive, the program must me capable of run with the new varaibles in the file each time.

2 Theoretical Framework

Processes and Threads A process is a program in execution with its own memory space. A thread is the smallest unit of execution inside a process. Threads within the same process share memory, making communication fast but requiring careful coordination. In this project, each vehicle runs as an independent POSIX thread (`pthread`).	**Concurrency and Race Conditions** Concurrency means multiple tasks overlap in time. When threads access shared data without coordination, the outcome depends on their arbitrary interleaving, producing a **race condition**. In the bridge, uncoordinated access could allow cars from opposite directions to enter simultaneously, violating the one-lane constraint.
Mutual Exclusion A **mutex** ensures only one thread executes a critical section at a time. A thread calls `pthread_mutex_lock()` to enter and `pthread_mutex_unlock()` to leave. All other threads attempting to lock it will block until it is free. The bridge uses a single global mutex to protect its shared state.	**Condition Variables and Starvation** A **condition variable** lets a thread sleep until a condition changes, avoiding busy-waiting. `pthread_cond_wait()` releases the mutex and suspends the thread; `pthread_cond_broadcast()` wakes all waiters. **Starvation**, where one side monopolizes the bridge, is prevented by the timed and quota-based turn-switching in the semaphore and police officer modes.

3 Explanation of the Solution

3.1 Architecture Overview

To better manangment and order we divided our projects into different files, each one with a unique rol:

- `main.c`: Loads configuration, initializes the bridge and visualization, and spawns two generator threads (one per side) that create car threads at exponentially distributed intervals.
- `puente.c`: Contains all synchronization logic, and the 3 modes of the brigde. Exposes `entrar_puente()`, `avanzar_este/oeste()`, and `salir_puente()` to car threads, basically the entire body of the project.
- `auto.c`: Defines the car thread function, which calls the bridge API in sequence and updates the visual state.
- `config.c`: Is the one responsible for loading the variables of the file as mentioned before, it parses the variables for the file and loads it into actual variables in the code.
- `visual.c`: The project had a "extra" assigment to make, a visual UI of the bridge. Visual.c is the one who manages all of the visual part via SDL2.

3.2 Bridge Synchronization

The core of the simulation is a single mutex protecting all shared bridge state: the number of cars currently on the bridge, the active direction, and the waiting queues per side and per vehicle type.

When a car thread calls `entrar_puente()`, it first calls `esperar_puente()`, which registers the car in the appropriate waiting queue and then enters a loop evaluating whether entry conditions are satisfied. If they are not, the thread calls `pthread_cond_wait()` and suspends. Every time a car exits via `salir_puente()`, a `pthread_cond_broadcast()` wakes all waiting threads so they can re-evaluate their conditions.

3.3 Bridge Traversal

The bridge is modeled as an array of cells protected by individual per-cell mutexes. A car advancing from west to east locks the next cell before unlocking its current one, ensuring that two cars never occupy the same cell simultaneously and that the visual state is always consistent. The `usleep()` call between steps controls the speed of each car.

3.4 Control Modes

Carnage requires no additional threads. The entry condition simply checks that the bridge is empty or that traffic is already flowing in the same direction.

Semaphore mode spawns a `semaforo_thread` that sleeps for the configured green duration, then acquires the mutex, flips the active direction, updates the visual, and broadcasts to wake waiting cars.

Police officer mode spawns a `policia_thread` that holds the mutex and uses `pthread_cond_timedwait()` to monitor the bridge state. When the current side's queue is empty and the other side has cars waiting, the thread starts a `secs_espera`-second countdown. If the situation is unchanged after the timeout, it calls `switch_turno()`, which updates the active turn, resets the car counter, updates the visual, and broadcasts. The thread also signals immediately whenever a new car arrives or the bridge empties, ensuring no unnecessary delays.

3.5 Ambulance Handling

Ambulances are tracked in separate counters per side. When an ambulance is waiting, normal cars on the same side are blocked from entering to avoid delaying the ambulance further. Ambulances bypass the turn and semaphore checks entirely: once the bridge is clear or already flowing in their direction, they enter immediately.

In police officer mode, when an ambulance crosses against the current turn, the turn variable is saved before the ambulance enters and restored once the bridge empties, so the officer's state is not corrupted by the ambulance's passage.

3.6 Visualization

The simulation includes a real-time graphical interface built with **SDL2** and **SDL2_ttf** for text rendering. The interface runs in a dedicated thread. The bridge is drawn as a single horizontal lane with per-cell car sprites that animate smoothly between positions. Waiting queues are rendered on both sides of the bridge, with ambulances visually distinct from regular cars. Each control mode adds its own overlay: traffic lights with red and green indicators for semaphore mode, and a turn display box for police officer mode. All shared visual state is protected by its own mutex, separate from the bridge mutex, to avoid blocking the simulation while the frame is being drawn.

4 Description of the Experience

4.1 Problems encountered

Our first major problem was related to how cars travel across the bridge. When we started the project, we initially thought that we only needed three threads: two for generating cars on each lane, respectively, and one for managing the bridge. In this approach, each car would simply travel across the bridge in a fixed amount of time. While this worked to some extent, the biggest issue arose when two cars in the same lane had different speeds. In that situation, one car cannot overtake the other, which became a significant concern. We resolved this issue after consulting the professor, who clarified what was actually expected.

Another anticipated challenge was implementing the UI. None of us had prior experience working in C, as we usually work with C++. Therefore, we had to learn how to build a UI in C from scratch. This was not only for the extra points but also because it made it much easier to track whether there were issues in the logical part of the project.

4.2 "Unique" Decisions

As mentioned before, we included a configuration file with several variables. While we were given some guidelines on what this file should contain, we decided to add additional parameters so that the "user" of the bridge could experiment more with the simulation. This allows for scenarios that would not be possible with only static variables.

Additionally, after discussing with friends from other groups, we noticed that some of our implementations were different, yet still valid. We want to highlight this as a reminder that, even when working on the same project with the same objectives, different perspectives can lead to different solutions. These differences should not be dismissed simply for being unconventional.

5 Personal Lessons Learned

5.1 Manfred's Lessons

The most impactful lesson I learned came from the biggest mistake we made: not consulting the professor before starting to code. Sometimes fixing a misunderstanding is straightforward, but other times the required change can be substantial enough to invalidate hours of work. From now on, I will make sure to fully clarify expectations before writing a single line.

As is usually the case with this type of project, time management proved to be a recurring challenge. As the deadline approached, I was reminded once again of how much room I have to improve in that area. On the positive side, the experience also pushed me to work better under pressure and to be more deliberate about how I plan my time.

Although I do not particularly enjoy coding in C, being forced to work with it gave me a genuine appreciation for everything that C++ adds on top. It is easy to take modern language features for granted until you spend time without them.

5.2 Jose's Lessons

I believe the most important thing I take away from this project is the ability to properly think through the logic behind managing the bridge and its different modes. While it might be more typical to highlight thread management, it is such a complex topic that I do not feel confident enough to say I have mastered it.

Another key takeaway is time management and how stressful it can be to handle multiple deadlines scheduled on consecutive days. Although challenging, this experience helped us improve our ability to cope with stress and better organize ourselves.

Finally, I would like to conclude by saying how much I enjoyed working on this type of project in retrospect. It once again reinforces my belief that I made the right choice when deciding my future career path.

6 Conclusions

This project successfully delivered a functioning narrow bridge simulation that correctly handles concurrent vehicle threads, enforces one-lane traffic, and supports three distinct control modes. Beyond meeting the functional requirements, the development process itself turned out to be as instructive as the final result.

From a technical standpoint, the project demonstrated that correct concurrent programming demands more than simply adding locks. The initial design, which treated the bridge as a passive resource managed by a single thread, had to be completely rethought once the per-car speed requirement was introduced. The final solution, where each car is its own thread and the bridge enforces access through a mutex and condition variables, is a more accurate model of how real operating systems coordinate competing processes. The police officer mode in particular required careful reasoning about ownership of shared state, as the interaction between the officer thread, car threads, and ambulance priority introduced subtle bugs that were only resolved by thinking rigorously about which thread is responsible for each piece of state at every point in time.

From a process standpoint, the most recurring theme across both teammates' experiences was the cost of unclear requirements. Starting to build before fully understanding what was expected led to significant rework. The lesson is simple but easy to forget under deadline pressure: time spent clarifying the problem upfront is never wasted.

Finally, the decision to invest in a real-time SDL2 visualization paid off beyond earning extra credit. Having a live view of the bridge made it possible to catch synchronization errors, such as cars stalling or turns not switching, that would have been extremely difficult to diagnose from console output alone. It reinforced a broader principle: visibility into a system's behavior is not a luxury but a debugging tool that directly reduces development time.